

# Projektarbeit

Semesterprojekt Master Informatik

# Implementierung von RISC-V-Extensions für den PicoNut-Prozessor

## A, Zkne/d-Extensions

erstellt von **Niklas Sirch**  
unter Mitwirkung von Daniel Sommerfeldt

**Niklas Sirch**

[illegible]

Matrikelnummer:

■■■■■■■■

|                |                          |
|----------------|--------------------------|
| Prüfer         | Prof. Dr. Gundolf Kiefer |
| Eingereicht am | 31.01.2026               |

**Technische Hochschule  
Augsburg**

An der Hochschule 1  
D-86161 Augsburg  
T +49 821 5586-0  
F +49 821 5686-3222  
www.tha.de  
info@tha.de

Hinweis: Der Umfang dieser Arbeit übersteigt wahrscheinlich den Anforderungen an eine Projektarbeit für Master Informatik. Also bitte nicht als Orientierung für den eigenen Arbeitsaufwand nehmen ;)

Diese Arbeit mit dem Titel  
„Implementierung von RISC-V-Extensions für den PicoNut-Prozessor“  
von  
Niklas Sirch  
unter Mitwirkung von Daniel Sommerfeldt  
steht unter einer  
[Creative Commons Attribution-Non Commercial-ShareAlike 4.0 International License \(CC BY-NC-SA\)](https://creativecommons.org/licenses/by-nc-sa/4.0/) .



Sämtliche, in der Arbeit beschriebene und auf dem beigelegten Datenträger vorhandene, Ergebnisse dieser Arbeit in Form von Quelltexten, Software und Konzeptentwürfen stehen unter einer BSD 2-Clause License.

BSD 2-Clause License

Copyright (c) 2026, THA Efficient Embedded Systems (EES) Group  
All rights reserved.

Redistribution and use in source and binary forms, with or without modification, are permitted provided that the following conditions are met:

1. Redistributions of source code must retain the above copyright notice, this list of conditions and the following disclaimer.
2. Redistributions in binary form must reproduce the above copyright notice, this list of conditions and the following disclaimer in the documentation and/or other materials provided with the distribution.

THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT HOLDER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

# Inhaltsverzeichnis

|          |                                                  |           |
|----------|--------------------------------------------------|-----------|
| <b>1</b> | <b>Einleitung</b>                                | <b>3</b>  |
| 1.1      | Motivation .....                                 | 3         |
| 1.2      | Ziele des Projekts .....                         | 3         |
| 1.3      | Aufbau der Arbeit .....                          | 3         |
| <b>2</b> | <b>Grundlagen</b>                                | <b>5</b>  |
| 2.1      | Das PicoNut-Projekt .....                        | 5         |
| 2.2      | RISC-V .....                                     | 6         |
| <b>3</b> | <b>A-Extension</b>                               | <b>7</b>  |
| 3.1      | Motivation für das PicoNut-Projekt .....         | 7         |
| 3.2      | Memory Consistency Model .....                   | 7         |
| 3.2.1    | High-Level Beschreibung von RVWMO .....          | 7         |
| 3.2.2    | Relevanz für das PicoNut-Projekt .....           | 8         |
| 3.3      | Atomic Memory Operations (Zaamo) .....           | 8         |
| 3.3.1    | Design und Anwendung .....                       | 8         |
| 3.3.2    | Implementierung .....                            | 10        |
| 3.4      | Load-Reserved / Store-Conditional (Zalrsc) ..... | 13        |
| 3.4.1    | Ablauf einer LR/SC-Sequenz .....                 | 14        |
| 3.4.2    | Anwendung von LR/SC .....                        | 14        |
| 3.4.3    | Implementierung .....                            | 15        |
| 3.5      | Ausblick A-Extension .....                       | 16        |
| <b>4</b> | <b>Skalare Kryptografie-Erweiterungen (AES)</b>  | <b>18</b> |
| 4.1      | Motivation für das PicoNut-Projekt .....         | 18        |
| 4.2      | Spezifikation .....                              | 18        |
| 4.3      | Advanced Encryption Standard .....               | 19        |
| 4.4      | AES-Instruktionen für RV32 .....                 | 19        |
| 4.4.1    | Funktionsweise .....                             | 20        |
| 4.4.2    | Implementierungsaspekte im PicoNut .....         | 20        |
| 4.5      | Software .....                                   | 21        |
| 4.6      | Ergebnisse .....                                 | 22        |
| 4.6.1    | Analyse .....                                    | 22        |
| 4.6.2    | Demonstrator .....                               | 22        |
| <b>5</b> | <b>RISCOF</b>                                    | <b>24</b> |
| 5.1      | Funktionsweise .....                             | 24        |
| 5.1.1    | Überblick .....                                  | 24        |

|                                         |           |
|-----------------------------------------|-----------|
| 5.1.2 Konfiguration .....               | 25        |
| 5.2 Integration .....                   | 25        |
| 5.2.1 RISCOF-PicoNut Plugin .....       | 25        |
| 5.2.2 Eigene Tests (Custom Tests) ..... | 25        |
| 5.3 Ergebnisse .....                    | 25        |
| 5.4 Ausblick .....                      | 26        |
| <b>6 Nebenaufgaben</b>                  | <b>27</b> |
| 6.1 ASCII-Tetris .....                  | 27        |
| 6.1.1 Softwareauswahl .....             | 27        |
| 6.1.2 Portierung .....                  | 27        |
| 6.2 GitLab CI Migration .....           | 28        |
| 6.2.1 Probleme mit Jenkins .....        | 28        |
| 6.2.2 Migration .....                   | 28        |
| 6.2.3 Verbesserungen .....              | 29        |
| 6.3 Video-Interface .....               | 29        |
| 6.3.1 Identifizierte Defizite .....     | 30        |
| 6.3.2 Wishbone Anbindung .....          | 30        |
| 6.3.3 Funktionserweiterungen .....      | 31        |
| 6.4 Weitere Tätigkeiten .....           | 32        |
| <b>7 Fazit</b>                          | <b>33</b> |
| 7.1 Zusammenfassung .....               | 33        |
| 7.2 Ausblick .....                      | 33        |
| <b>I Abbildungsverzeichnis</b>          | <b>34</b> |
| <b>II Quellcodeverzeichnis</b>          | <b>35</b> |
| <b>III Literaturverzeichnis</b>         | <b>36</b> |
| <b>Erklärung zur Projektarbeit</b>      | <b>38</b> |

# 1 Einleitung

---

RISC-V ist eine offene Befehlssatzarchitektur (Instruction Set Architecture, ISA). Studierende der Technischen Hochschule Augsburg (THA) in der Forschungsgruppe Effiziente Eingebettete Systeme (EES) entwickeln im Rahmen des PicoNut-Projekts einen minimalistischen RISC-V-Prozessor, der als Lehr- und Forschungsplattform dient. Dieser Bericht dokumentiert die Erweiterung des PicoNut-Prozessors um zusätzliche RISC-V ISA-Extensions. Die Arbeit ist zweigeteilt in die Implementierung der A-Extension für atomare Speicheroperationen und die Zkne- und Zknd-Extensions für Advanced Encryption Standard (AES) Instruktionen.

## 1.1 Motivation

Die Weiterentwicklung des PicoNut-Prozessors zielt darauf ab, die Funktionalität und Anwendbarkeit des Systems zu erhöhen. Durch die Implementierung der A-Extension wird die notwendige Hardware-Basis geschaffen, um Multi-Core-Synchronisation zu ermöglichen und komplexe Betriebssysteme wie Linux lauffähig zu machen. Parallel dazu öffnet die Integration der skalaren Kryptografie-Erweiterungen einen Einblick in die Kryptografie für Studierende. Da Software-Lösungen oft rechenintensiv und anfällig für Angriffe sind, bietet die Hardware-Beschleunigung von AES einen signifikanten Performance- und Sicherheitsgewinn. Die begleitende Modernisierung der Infrastruktur und Verifikation stellt dabei sicher, dass das System trotz steigender Komplexität stabil bleibt und offiziellen RISC-V-Standards entspricht.

## 1.2 Ziele des Projekts

Das Hauptziel ist die funktionale Erweiterung des bestehenden RV32I-Kerns:

- Implementierung der A-Extension: Integration von Zaamo und Zalrsc in den Nucleus und die Membrana zur Unterstützung von Multi-Core-Primitiven.
- Hardware-AES: Beschleunigung der Verschlüsselung durch dedizierte Instruktionen zur Steigerung der Performance (Ziel: deutlicher Speedup gegenüber Software-Bibliotheken).
- Verifikation: Sicherstellung der Standardkonformität mittels des RISCOF-Frameworks.
- Infrastruktur: Modernisierung der CI/CD-Pipeline und Anbindung des Video-Interface.

## 1.3 Aufbau der Arbeit

Die vorliegende Arbeit gliedert sich wie folgt: Zunächst werden in [Abschnitt 2](#) die wesentlichen Grundlagen erläutert. Dies umfasst eine Vorstellung des PicoNut-Projekts sowie eine Einführung in die offene Befehlssatzarchitektur RISC-V.

In [Abschnitt 3](#) wird detailliert auf die Implementierung der RISC-V A-Extension eingegangen. Hierbei werden die theoretischen Konzepte der atomaren Speicheroperationen (Zaamo) und der Load-Reserved/Store-Conditional-Mechanismen (Zalrsc) sowie deren konkrete hardwareseitige Umsetzung im PicoNut-Nucleus und der Membrana beschrieben.

Anschließend behandelt [Abschnitt 4](#) die skalaren Kryptografie-Erweiterungen. Der Fokus liegt hierbei auf der Beschleunigung des Advanced Encryption Standard (AES) durch spezifische Instruktionen und deren Auswirkungen auf die Performance und Sicherheit des Systems.

Um die Korrektheit dieser Erweiterungen sicherzustellen, setzt sich [Abschnitt 5](#) mit dem Compliance-Framework RISCOF auseinander. Es wird aufgezeigt, wie dieses Werkzeug Konformität sicherstellt und in den bestehenden Entwicklungsprozess integriert wurde.

[Abschnitt 6](#) widmet sich verschiedenen Nebenaufgaben, die im Zuge des Projekts angefallen sind. Dazu zählen die Portierung eines ASCII-Tetris-Spiels für Demonstrationszwecke, die Migration der Continuous Integration auf GitLab CI sowie die Anbindung extern entwickelter Video-Interfaces an den PicoNut-Prozessor.

Abschließend fasst [Abschnitt 7](#) die Ergebnisse der Arbeit zusammen und gibt einen Ausblick auf mögliche zukünftige Entwicklungen im Bereich der RISC-V-Extensions.

---

|                                                                    |                                                          |
|--------------------------------------------------------------------|----------------------------------------------------------|
| Interner develop Commit-Hash<br>mit allen Änderungen dieser Arbeit | <a href="#">5b54426a0fac175587174536919a09b8c5174279</a> |
| Voraussichtlich veröffentlichte Version                            | > 1.1.0                                                  |

## 2 Grundlagen

---

Dieses Kapitel erläutert die wesentlichen Grundlagen, die für das Verständnis der Entwicklung von RISC-V-Extensions für den PicoNut-Prozessor im Rahmen dieser Arbeit notwendig sind.

### 2.1 Das PicoNut-Projekt

Das PicoNut-Projekt verfolgt das Ziel, modulare RISC-V-Systeme in SystemC zu entwickeln. Ein solches System besteht aus einem Prozessor, einem Systembus und der entsprechenden Peripherie. Als Systembus wird derzeit der „Wishbone“-Standard verwendet.

Das Projekt nutzt den Intel Compiler for SystemC (ICSC), um SystemC-Code in die Sprache SystemVerilog zu übersetzen, welche anschließend für die Synthese auf FPGAs verwendet werden kann [1]. Aktuell wird hierfür das ULX3S-Board mit einem Lattice-FPGA eingesetzt.

Der Fokus dieser Arbeit liegt auf dem Prozessor, dessen Aufbau in [Abbildung 1](#) dargestellt ist. Der Prozessor ist modular aufgebaut und besteht aus der Ausführungseinheit „Nucleus“ sowie der „Membrana“, welche die Systembusanbindung und die Load-Store-Unit umfasst. Einzelne Module können flexibel ausgetauscht werden, um unterschiedliche Funktionalitäten bereitzustellen oder um in der Simulation performantere Implementierungen zu nutzen.

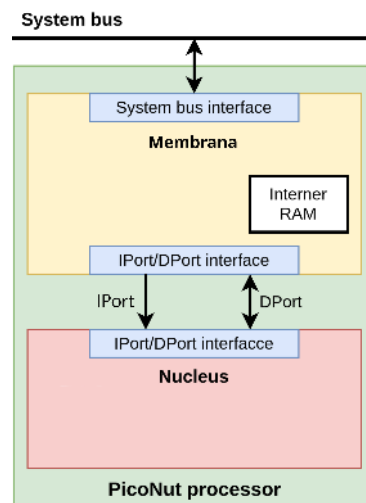


Abbildung 1: Aufbau eines PicoNut-Prozessorkerns

Momentan existiert mit dem „Referenz-Nucleus“ eine Basisimplementierung, die ohne Pipelining arbeitet, nur Single-Core-fähig ist und den 32-Bit RISC-V-Befehlssatz unterstützt.

Für die Membrana stehen zwei Varianten zur Verfügung: „Membrana-HW“, die synthetisierbar ist, und „Membrana-Sim“, die für eine höhere Simulationsgeschwindigkeit besitzt und Soft-Peripherie unterstützt.

## 2.2 RISC-V

RISC-V ist eine offene Befehlssatzarchitektur (ISA), die in den letzten Jahren stark an Popularität gewonnen hat. Im Gegensatz zu Architekturen wie ARM kann RISC-V ohne Lizenzgebühren genutzt und erweitert werden, während sie gleichzeitig eine breite Unterstützung durch die Community und Industrie genießt [2].

Die Architektur basiert auf dem Prinzip des Reduced Instruction Set Computer (RISC). Der Befehlssatz ist bewusst einfach gehalten, um eine effiziente Implementierung zu ermöglichen. Um Beschleunigungen oder zusätzliche Funktionen zu bieten, sind neben dem Basisbefehlssatz (I) verschiedene optionale Erweiterungen spezifiziert. Bekannte Beispiele hierfür sind die Erweiterungen für Multiplikation und Division (M), Gleitkommaoperationen (F) oder atomare Operationen (A) [3].

## 3 A-Extension

---

Die RISC-V A-Extension ist eine Erweiterung der RISC-V-Architektur, die atomare Operationen sowie Compare-and-Swap (CAS) in Multi-Core-Prozessoren ermöglicht [4].

Die A-Extension lässt sich in zwei Teilbereiche gliedern:

1. Zaamo für Atomic Memory Operations (AMOs)
2. Zalrsc für Load-Reserved/Store-Conditional (LR/SC)

### 3.1 Motivation für das PicoNut-Projekt

Die Implementierung der A-Extension im PicoNut-Projekt beruht auf mehreren Überlegungen. Zum einen bildet sie eine essenzielle Grundlage zur Unterstützung von Multi-Core, wie in den folgenden Abschnitten erläutert wird.

Zum anderen sind atomare Operationen für Betriebssysteme wie Linux unabdingbar, insbesondere für die Implementierung von Synchronisationsmechanismen wie Futex (Fast Userspace Mutexes) oder für speicherübergreifende Operationen zwischen CPU-Kernen [5]. Auch Benutzerbibliotheken wie Pthreads setzen auf diese Mechanismen, um zuverlässige Synchronisation zu gewährleisten.

Daher ist die A-Extension eine wichtige Voraussetzung für die Ausführung moderner Software auf PicoNut.

### 3.2 Memory Consistency Model

Bei der Implementierung hardwarebasierter Synchronisation ist das zugrunde liegende Memory Consistency Model des Prozessors entscheidend. Es existieren verschiedene Modelle, wie etwa **Sequential Consistency** (SC) und **Release Consistency** (RC) [6]. Diese Modelle unterscheiden sich darin, wie und wann Speicheroperationen in Multi-Core-Systemen sichtbar werden und beeinflussen somit die Korrektheit und Performance von Synchronisationsmechanismen.

Im folgenden Abschnitt wird das von RISC-V verwendete Konsistenzmodell beschrieben und dessen Relevanz für das PicoNut-Projekt erläutert.

#### 3.2.1 High-Level Beschreibung von RVWMO

RISC-V verwendet das RISC-V Weak Memory Ordering (RVWMO) Modell, das auf dem Konzept der **Release Consistency** basiert [3].

Dieses schwache Konsistenzmodell erlaubt es, dass Speicheroperationen in einer anderen Reihenfolge sichtbar werden, als sie im Quellcode erscheinen. Diese Flexibilität ermöglicht Hardwareoptimierungen, die die Performance steigern [7].

Zur Steuerung der Sichtbarkeit von Speicheroperationen in Bezug auf AMOs können FENCE-Instruktionen verwendet werden. Zusätzlich gibt es zwei wichtige Flags, die das Verhalten einzelner AMO-Instruktionen beeinflussen:

- **aq (acquire)**: Garantiert, dass keine spätere Speicheroperation vor der Instruktion sichtbar ist.
- **rl (release)**: Garantiert, dass alle vorherigen Speicheroperationen sichtbar sind, bevor die Instruktion ausgeführt wird.

Sind beide Flags gleichzeitig gesetzt, wird sequentielle Konsistenz garantiert.

### 3.2.2 Relevanz für das PicoNut-Projekt

Der aktuelle PicoNut-Prozessor ist ein Single-Core Prozessor ohne Caches oder Pipelining. Daher ist das Memory Consistency Model derzeit nicht relevant für seine Funktion.

Für die A-Extension wurde dennoch geprüft, ob bereits jetzt eine Berücksichtigung des Konsistenzmodells sinnvoll wäre. Es wurde sich dagegen entschieden, da:

- Es im aktuellen Stand keine funktionale Notwendigkeit gibt.
- Eine frühzeitige Implementierung des Modells den Umfang der Projektarbeit deutlich überschreiten würde.

## 3.3 Atomic Memory Operations (Zaamo)

Im Folgenden werden Atomic Memory Operations (AMOs) beschrieben, wie sie im PicoNut-Prozessor implementiert sind.

### 3.3.1 Design und Anwendung

Die Spezifikation der A-Extension [4] definiert neun AMO-Instruktionen. Diese umfassen arithmetische (z.B. AMOADD) und logische Operationen (AMOAND, AMOOR, AMOXOR), Min-/Max-Operationen (signed und unsigned), sowie eine reine Austauschoperation (AMOSWAP).

Die AMO-Instruktionen können allgemein als fetch-and-op Instruktionen beschrieben werden. Obwohl alle Operationen der Extension auch durch Load-Reserved und Store-Conditional (LR/SC) implementiert werden könnten, hat sich RISC-V für die AMO-Instruktionen entschieden, um für „hoch-parallele Workloads“ eine bessere Performance zu bieten [4].

#### Ablauf einer AMO-Instruktion

1. Der Wert an der Speicheradresse aus Register *rs1* wird geladen.
2. Der geladene Wert wird in das Zielregister *rd* geschrieben.
3. Der geladene Wert wird mit dem Inhalt von *rs2* entsprechend der gewählten AMO-Operation verarbeitet (z.B. swap, add, min).
4. Das Ergebnis wird an die ursprüngliche Speicheradresse (aus *rs1*) zurückgeschrieben.

Die Codierung dieser Instruktionen ist in [Abbildung 2](#) dargestellt.

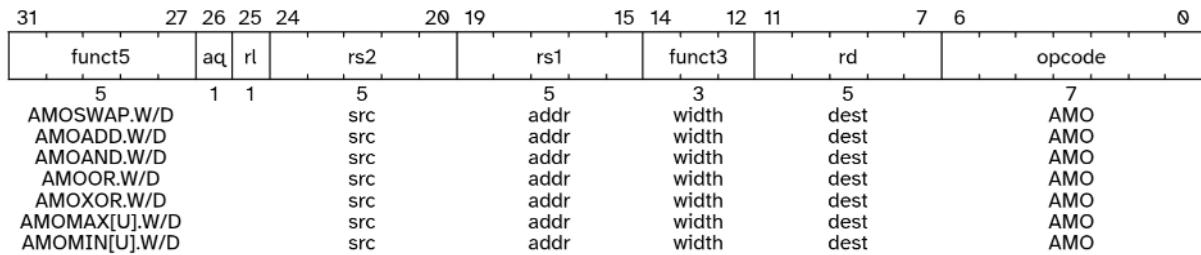


Abbildung 2: Codierung der AMO-Instruktionen

**Anwendung** Bei den Operationen hat man sich zum einen an den C/C++ Standards von 2011 orientiert. Zum anderen wollte man aber auch Operationen mit Peripherie-Registern vereinfachen.

C/C++11 fügt den Sprachen den `stdatomic.h` Header hinzu, der den neuen low-level Datentyp-Modifizierer `_Atomic` einführt. Diese Modifizierer gibt dem Compiler die Information, dass auf die Variable atomar zugegriffen werden muss. In RISC-V werden daher AMO-Instruktionen verwendet, um Operationen auf diesen Variablen durchzuführen. (Leider gibt es keine Warnung vom Compiler, wenn eine Operation nicht unterstützt wird.)

Eine beispielhafte Anwendung von `stdatomic.h` ist ein einfacher Zähler, der von mehreren Threads inkrementiert wird (Listing 1).

```

_Atomic int atomic_counter = 0;

int add_to_counter() {
    for (int i = 0; i < ITERATIONS; i++) {
        atomic_fetch_add(&atomic_counter, 1); // will be compiled to amoadd
    }
    return 0;
}

for (int i = 0; i < NUM_THREADS; i++) {
    pthread_create(&threads[i], NULL, add_to_counter, NULL);
}

```

Listing 1: Beispiel für die Verwendung von `stdatomic.h` zur Implementierung eines atomaren Zählers

Die von der Spezifikation gegebene Beispielanwendung für AMOs ist die Implementierung eines Spinlocks (Listing 2). Hierbei wird die `amoswap`-Instruktion verwendet, um den Lock-Wert atomar zu setzen und zu überprüfen. Die Applikation weiß, dass der Spinlock erfolgreich erworben wurde, wenn im Register `t1` (Argument für `rd`), das Zielregister für den alten Wert, der Speicherstelle `a0` (Argument für `rs1`), der Wert `0` zurückgegeben wird. Wenn dies nicht der Fall ist, weiß die Applikation, dass der Lock bereits von einem anderen Thread gehalten wird, und versucht es erneut.

Praktisch dabei ist, dass dies auch funktioniert, wenn mehrere Kerne gleichzeitig versuchen, den Spinlock zu erwerben, da die amoswap-Instruktion atomar ist.

```
li t0, 1 # Initialize swap value.
again:
lw t1, (a0) # Check if lock is held.
bnez t1, again # Retry if held.
amoswap.w.aq t1, t0, (a0) # Attempt to acquire lock.
bnez t1, again # Retry if held.
# ...
# Critical section.
# ...
amoswap.w.rl x0, x0, (a0) # Release lock by storing 0.
```

RV32 Assembly

Listing 2: Beispiel für die Verwendung von AMOs zur Implementierung eines Spinlocks (von [4])

### 3.3.2 Implementierung

Im Weiteren wird die Implementierung der A-Extension im PicoNut-Prozessor an Nucleus und Membrana beschrieben.

**Nucleus** Die Ausführung einer Instruktion stellte für den bisherigen Nucleus eine Herausforderung dar, da dieser nicht darauf ausgelegt war beispielsweise Zwischenergebnisse zu speichern. In [Abbildung 3](#) ist die Struktur des alten Nucleus ohne A-Extension dargestellt. Diese Struktur ist für RV32I-Instruktionen optimiert.

Betrachtet man den Ablauf einer einzelnen Instruktion ([Seite 8](#)), so ist in Schritt 1 eine Speicherung des Inhalts von rs1 erforderlich. Dabei darf jedoch keines der 31 Register verwendet werden, da diese für nachfolgende Instruktionen benötigt werden.

Obwohl die Spezifikation in Schritt 2 eine Speicherung des alten Wertes vom Inhalt von rs1 in rd ist, darf dies erst nach der Berechnung der AMO-Operation erfolgen, da rs2==rd gelten kann. Diese Art von Instruktion ist mit der riscv-gnu-toolchain sogar sehr häufig, da sich der Compiler ein Register einspart.

Zudem darf die Zieladresse des Datenports nicht immer mit einem Zwischenergebnis der Instruktion addiert werden.

Um die Operationen der AMOs zu ermöglichen, muss die Struktur erweitert werden. Dafür werden neue Multiplexer für die A-Extension hinzugefügt.

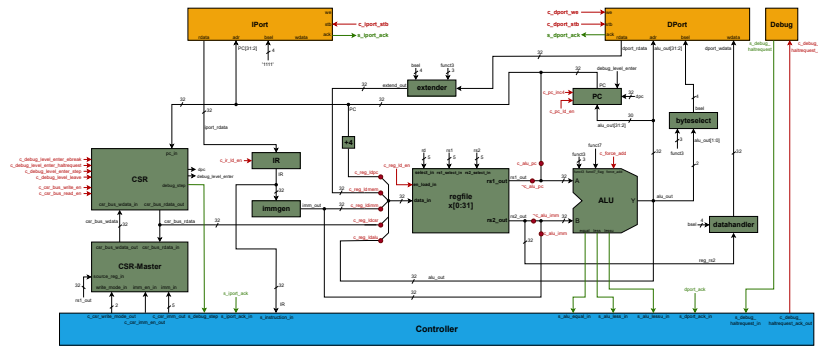


Abbildung 3: Nucleus ohne A-Extension

Das Problem der Zwischenspeicherung wurde zunächst mit mehreren Ansätzen untersucht. Eine Möglichkeit bestand darin, zusätzliche Register zur Zwischenspeicherung einzusetzen. Dies führte jedoch zu einer erhöhten Komplexität und einem größeren Flächenbedarf im Nucleus.

Schließlich erinnerte Gundolf Kiefer an die PicoNut-Architekturregel „grundsätzlich Register an Ausgängen“. Diese Regel war bereits umgesetzt, jedoch nicht dokumentiert. Mithilfe der bereits vorhandenen Register an Ein- und Ausgängen konnte das Problem der Zwischenspeicherung durch ein zusätzliches Enable Signal an den Registern gelöst werden. Diese Lösung sollte auch für zukünftige Erweiterungen der Architektur funktionieren. In [Abbildung 4](#) sind diese Register in **gelb** hervorgehoben.

Um den komplexeren Datenfluss zu unterstützen, wurden mehrere Multiplexer hinzugefügt – beispielsweise für das Schreiben in den Speicher oder die Bereitstellung der Operanden.

Zur Unterstützung der AMO-Operationen wurden zusätzliche ALU-Steuerleitungen ergänzt. Diese bestimmen die Operation, die von der ALU ausgeführt wird, basierend auf dem in funct5 kodierten Wert (funct5 ist im funct7-Feld der Instruktion enthalten).

In [Abbildung 4](#) ist die erweiterte Struktur des Nucleus mit der A-Extension dargestellt.

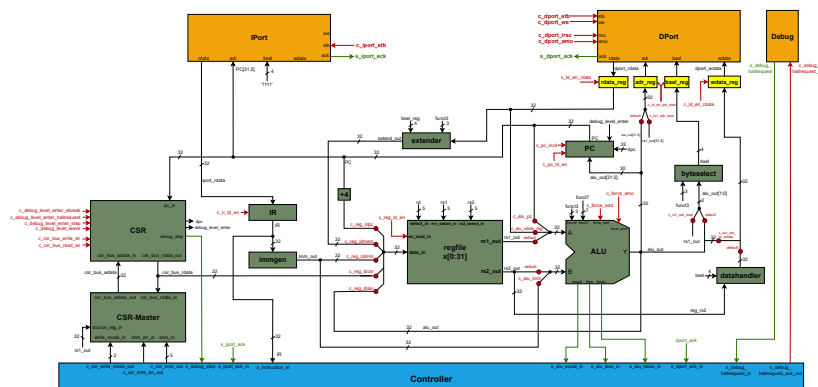


Abbildung 4: Nucleus mit A-Extension

**Controller** Der Controller ist das Steuerwerk des PicoNut-Nucleus. Er steuert den Ablauf der Instruktionen durch das Setzen von Steuersignalen. Der Controller basiert auf einem Moore-Au-



zu Atomaren Operationen anderen Nuclei weiterzugeben. Dazu wird die Membrana verwendet, die als zentrale Load/Store-Einheit und Systembus Schnittstelle diese Operationen koordinieren muss.

Die Anforderung der AMO-Instruktionen während einer Operation ist, dass kein anderer Core während der Ausführung auf die betroffene Speicheradresse zugreifen oder schreiben darf.

Daher muss ein Nucleus beim Ausführen der AMO-Instruktion sofort der Membrana mitteilen, dass eine atomare Operation ausgeführt wird. Unsere Umsetzung dafür ist ein neues Signal zwischen Nucleus und Membrana für atomare Lesen und Schreiben: `dport_amo`.

Beim Lesen der Adresse für den AMO wird dieses Signal gesetzt. Die Membrana merkt sich damit, dass für diese Adresse eine atomare Operation ausgeführt wird. Solange diese Operation nicht abgeschlossen ist, werden alle anderen Zugriffe auf diese Adresse von anderen Nuclei blockiert indem diese Zugriffe erst nach der AMO ausgeführt werden. Beim Schreibe-Teil der AMO wird das Signal wieder gesetzt, was der Membrana signalisiert, dass die atomare Operation abgeschlossen ist und die Adresse wieder für andere Nuclei freigegeben werden kann.

Im SystemC Code wird das Merken der blockierten Adressen in der Klasse/Struktur `c_memu_a_ext` realisiert. Für jeden Core wird dabei die Adresse und ein Flag gespeichert, ob eine AMO ausgeführt wird. In der State Machine der Membrana wird bei jedem Lese- und Schreibzugriff überprüft, ob die Adresse von einer AMO betroffen ist.

Theoretisch wäre es auch möglich gewesen, während einer AMO den gesamten Speicherzugriff für andere Nuclei zu blockieren. Dies würde Hardware und Zeit für die Vergleiche der Adressen einsparen. Wie aber in [Abschnitt 3.4](#) beschrieben, werden für Load-Reserved/Store-Conditional eine ähnliche Strukturen benötigt. Was den Zeitaufwand für Vergleiche nicht umgehbar macht.

Die Struktur `c_memu_a_ext` für die Implementierung zu benutzen war eigentlich nicht vorgesehen, da Strukturen in Signalen nicht üblich sind. Jedoch hatte das Synthesis-Tool Yosys Probleme bei der Initialisierung von Arrays von Signalen.

### 3.4 Load-Reserved / Store-Conditional (Zalrsc)

*Autor: Daniel Sommerfeldt*

Die Zalrsc-Extension erweitert RISC-V um die beiden Instruktionen LR (Load-Reserved) und SC (Store-Conditional). Zusammen stellen sie einen Mechanismus bereit, um komplexere atomare Operationen auf Speicherwörter zu implementieren. Sie gelten als „primitive“ Atomics, auf denen höhere Synchronisationsmechanismen wie Compare-and-Swap (CAS), Spinlocks oder lock-freie Datenstrukturen aufbauen können.

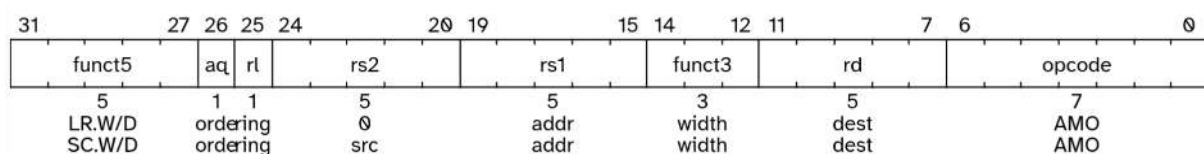


Abbildung 7: Codierung der lr/sc-Instruktionen

Während AMO-Instruktionen („fetch-and-op“) fest vorgegebene atomare Operationen in einer einzigen Instruktion ausführen, erlaubt das LR/SC-Paar flexible, zusammengesetzte atomare Abläufe.

### 3.4.1 Ablauf einer LR/SC-Sequenz

Eine LR/SC-Sequenz umfasst immer genau eine LR-Instruktion und eine SC-Instruktion in Program Order.

#### 1. LR $rs1 \rightarrow rd$

1. Aus der Adresse in  $rs1$  wird ein Wort (W) oder Doppelwort (D) geladen.
2. Der Wert wird in  $rd$  geschrieben (bei RV64: Wort  $\rightarrow$  sign-extended).
3. Die Implementierung registriert eine **Reservation Set**, das alle Bytes umfasst, die im späteren SC getestet werden.

**2. Zwischenrechnung** Zwischen LR und SC kann die Software beliebige Berechnungen durchführen.

#### 3. SC $rs2 \rightarrow (rs1)$

1. Der Prozessor prüft, ob keine fremden Stores im Reservation Set beobachtet wurden.
2. Falls die Reservation noch gültig ist:
  - Der Wert aus  $rs2$  wird atomar an die Adresse in  $rs1$  geschrieben.
  - $rd$  wird auf 0 gesetzt.
3. Falls die Reservation ungültig wurde:
  - Es wird nicht geschrieben.
  - $rd$  wird ungleich 0 gesetzt, um den Fehlschlag anzuzeigen.

Nach **jedem** SC – egal ob erfolgreich oder nicht – wird die Reservation ungültig.

### 3.4.2 Anwendung von LR/SC

Während AMOs für häufige arithmetische und logische Operationen optimiert sind, bieten die Instruktionen `lr` (Load Reserved) und `sc` (Store Conditional) die notwendige Flexibilität, um beliebige Read-Modify-Write (RMW) Sequenzen zu implementieren. Das bekannteste Beispiel hierfür ist die „Compare-and-Swap“ (CAS) Operation, die die Grundlage für viele komplexe, sperrfreie (lock-free) Datenstrukturen bildet.

Die `lr`-Instruktion lädt einen Wert und registriert eine „Reservierung“ auf der Speicheradresse. Die darauf folgende `sc`-Instruktion schreibt einen neuen Wert nur dann erfolgreich in die Speicheradresse, wenn die Reservierung noch gültig ist (d.h. wenn kein anderer Hart in der Zwischenzeit an diese Adresse geschrieben hat).

Ein entscheidender Unterschied zu AMOs ist, dass der Programmierer hier selbst für die Erfolgskontrolle verantwortlich ist. Schlägt das `sc` fehl (Rückgabewert ungleich 0), muss die Operation in einer Schleife wiederholt werden ([Listing 3](#)).

|                                                  |               |
|--------------------------------------------------|---------------|
| # a0: Address, a1: Expected Value, a2: New Value | RV32 Assembly |
|--------------------------------------------------|---------------|

```

cas:
    lr.w t0, (a0)      # Load original value and reserve.
    bne t0, a1, fail   # Check if value matches expectation.
    sc.w t1, a2, (a0)  # Attempt to write new value.
    bnez t1, cas       # Retry if reservation failed (t1 != 0).
    li a0, 0           # Success (return 0).
    ret
fail:
    li a0, 1           # CAS failed (value didn't match).
    ret

```

Listing 3: Implementierung einer Compare-and-Swap (CAS) Funktion mittels LR/SC

Ein kritischer Aspekt bei der Verwendung von LR/SC ist die Einhaltung der architektonischen Einschränkungen für den Code zwischen dem `lr` und dem `sc`. Die RISC-V Spezifikation schreibt vor, dass diese Sequenz kurz sein muss (in der Regel maximal 16 Instruktionen), keine Speicherzugriffe oder Sprünge enthalten darf und keine Ausnahmen auslösen sollte.

Werden diese Regeln nicht eingehalten, kann die Lebendigkeit (Liveness) des Systems nicht garantiert werden, was im schlimmsten Fall zu einem Livelock führt, bei dem die Reservierung immer wieder fehlschlägt. Der Compiler (z.B. bei Verwendung von `atomic_compare_exchange` in C) kümmert sich in der Regel automatisch um die Erzeugung dieser korrekten Sequenzen.

### 3.4.3 Implementierung

**Controller** Die Unterstützung der Instruktionen `LR.W/D` und `SC.W/D` erfordert eine Erweiterung der Zustände des Controllers.

Die Zustandsabfolge `LR1`, `LR2` und `LR3` ist analog zum regulären Ladeprozess (`LOAD1`, `LOAD2`, `LOAD3`) konzipiert. Der funktionale Unterschied zeigt sich im Zustand `LR2`, in welchem die Signalleitung `dp_lr_sc` aktiviert wird. Dieses Signal fungiert als Flag, welches die Membrana anweist, die notwendigen Prozesse zur Aufnahme der Adresse in ein Reservation Set einzuleiten.

Der Prozess des bedingten Schreibens wird durch die Zustände `SC1`, `SC2` und `SC3` abgebildet, welche strukturell den unbedingten Schreiboperationen (`STORE1`, `STORE2`, `STORE3`) entsprechen. Auch in dieser Sequenz wird das Steuersignal `dp_lr_sc` gesetzt, um die Validierung der Reservierung zu initiieren. Es ist hervorzuheben, dass der Controller selbst keinen dedizierten Fehlerzustand implementiert. Die Behandlung eines fehlgeschlagenen Schreibversuchs obliegt vollständig der Membrana. Der Status der Operation wird transparent über den Data-Port zurückgemeldet, wobei ein Wert von 1 auf der `rdata`-Leitung das Scheitern der Reservierungsprüfung indiziert.

**Membrana** Analog zu den Zaamo-Befehlen erfordert die Speicherschnittstelle (Membrana) eine funktionale Erweiterung. Es muss ein Mechanismus implementiert werden, der in der Lage ist, einen fehlgeschlagenen Schreibversuch auf einen nicht validen reservierten Speicherbereich

zu signalisieren. Der technische Lösungsansatz unterscheidet sich dabei zwischen den beiden Membrana-Varianten `membrana_soft` und `membrana_hw`.

**membrana\_soft** Für die simulationsbasierte Membrana (Sim-Only) ist das Anlegen eines Reservation Sets pro Prozessorkern ausreichend. In diesem Array, mit einer aktuellen Breite von 32-Bit, wird bei jedem Aufruf eines LR-Befehls die angeforderte Adresse `addr` hinterlegt. Dies ermöglicht es jedem CPU-Kern, eine Adresse individuell als reserved zu markieren.

Jeder nachfolgende Schreibzugriff initiiert einen Vergleich mit allen Einträgen im Reservation Set. Wird eine Übereinstimmung mit der zu beschreibenden Adresse festgestellt, wird der entsprechende Eintrag invalidiert (aus dem Array gestrichen). Ist das `lrsc`-Flag gesetzt, erfolgt eine Validierung, ob sich die zuvor reservierte Adresse noch im Reservation Set befindet. Da jeder Schreibzugriff, unabhängig davon ob er durch SC oder reguläre Befehle wie SW ausgelöst wurde, das Set prüft und Einträge entfernt, garantiert eine bestehende Reservation, dass der Wert an der Adresse seit dem letzten LR unverändert ist. Ist die Reservation gültig, verläuft der Schreibprozess regulär. Andernfalls wird der Schreibvorgang unterbunden und `rdata` mit dem Fehlercode 1 beschrieben.

**membrana\_hw** Die Umsetzung in der synthetisierbaren Membrana folgt demselben logischen Prinzip, erfordert jedoch mehr Arbeit in der internen Zustandsmaschine. Hierbei kommen die zusätzlichen Zustände `dp_read_lr`, `write_check`, `write_sc_success` und `write_sc_fail` zum Einsatz.

Während der Hardware-Implementierung des Reservation Sets traten spezifische Fehler im Synthese-Tool Yosys auf. Diese resultierten vermutlich aus Problemen bei der Überprüfung der Bitbreiten verschiedener Register. Durch die Kapselung der Logik in eine separate Klasse konnte dieser Fehler behoben werden, auch wenn die exakte Ursache im Tool unbekannt ist.

Bei aktivem `dp_lr_sc`-Flag wechselt die Zustandsmaschine nach dem Zustand `dp_read_check` in den Zustand `dp_read_lr`. In diesem Zustand wird das Reservation Set aktualisiert, bevor der Prozess im regulären Lesezustand `dp_read_rdata` fortgesetzt wird.

Ein Schreibzugriff mit gesetztem `dp_lr_sc`-Flag löst analog zur Sim-Only Membrana eine Überprüfung des Reservation Sets aus. Basierend auf dem Ergebnis wird entschieden, ob in den Zustand `write_sc_success` oder `write_sc_fail` verzweigt wird. Wie die Bezeichnungen implizieren, führt `write_sc_success` den regulären Schreibprozess aus, während `write_sc_fail` den Vorgang abbricht und den Wert `dp_rdata` auf 1 setzt.

## 3.5 Ausblick A-Extension

*Autor: Daniel Sommerfeldt*

Obwohl die Implementierung der A-Extension einen signifikanten Fortschritt für die allgemeine Software-Unterstützung des PicoNut darstellt, bietet sie in der aktuellen Konfiguration des Nucleus nur begrenzte unmittelbare Vorteile. Der primäre Verwendungszweck dieser Erweiterung liegt in der Realisierung atomarer Zugriffskontrollen in Multicore-Systemen. Da der PicoNut derzeit im Single-Core-Modus betrieben wird, laufen A-Extension-Befehle faktisch immer erfolgreich ab. Es existieren keine parallelen Speicherzugriffe, die ein Reservation Set eines anderen Kerns invalidieren oder einen laufenden AMO-Befehl blockieren könnten. Dennoch

wurde die A-Extension unter der Prämisse einer zukünftigen Skalierung auf Multicore-Systeme entwickelt und enthält bereits alle notwendigen Schnittstellen für eine nahtlose Integration.

Zudem ermöglichen die nun unterstützten Befehle die Ausführung von Software, die mit dem Compiler-Flag `-march=rv32ia` übersetzt wurde. Hervorzuheben ist hierbei eine Linux-Variante, die einen wichtigen Meilenstein in der Entwicklung des PicoNut markiert. Der Kernel nutzt die A-Extension zur Optimierung in Multicore-Umgebungen. Auch wenn diese Optimierung im Single-Core-Betrieb technisch nicht zwingend erforderlich ist, ist die Unterstützung der vom Compiler generierten Instruktionen für die Kompatibilität unerlässlich.

Die A-Extension ist ferner Bestandteil der RV32G-Instruktionsgruppe, welche als „General-Purpose“-Standard deklariert ist.

## 4 Skalare Kryptografie-Erweiterungen (AES)

---

Die Sicherheit von Datenübertragung und -speicherung ist in modernen eingebetteten Systemen von zentraler Bedeutung. Da Software-Implementierungen kryptografischer Algorithmen oft rechenintensiv und anfällig für Seitenkanalangriffe sind, spezifiziert die RISC-V Foundation die „Scalar Cryptography Extensions“.

Im PicoNut-Projekt wurden zu diesem Zweck Erweiterungen für die AES-Verschlüsselung implementiert.

### 4.1 Motivation für das PicoNut-Projekt

Die Integration kryptografischer Erweiterungen in den PicoNut-Prozessor dient primär dem Lernen im akademischen Kontext. Aber auch in der Praxis sind solche Erweiterungen sehr relevant, da Kryptografie in eingebetteten Systemen zunehmend an Bedeutung gewinnt – beispielsweise durch die verpflichtende Verschlüsselung auf IoT-Geräten im Zuge von Regelungen wie dem Cyber Resilience Act der EU [8]. Daher ist es für das Projekt vorteilhaft, über eine eigene Implementierung von Verschlüsselungsalgorithmen zu verfügen.

In dieser Arbeit liegt der Fokus auf der performanten Implementierung des Advanced Encryption Standard (AES). Es existieren jedoch noch viele weitere relevante Themenfelder der Kryptografie, wie etwa Hashfunktionen oder die Resistenz gegen Seitenkanalangriffe, die in zukünftigen Arbeiten behandelt werden können.

### 4.2 Spezifikation

RISC-V spezifiziert verschiedene kryptografische Erweiterungen in Unprivileged ISA-Spezifikationen [3], die in Skalare- und Vektor-Kryptografie unterteilt sind. Dabei bauen Vektor-Kryptografie auf der V-Extension auf und nutzt die Vektorregister für parallele Operationen. Skalare Kryptografie hingegen operiert auf den allgemeinen Registern. Das PicoNut-Projekt implementiert im Moment keine Vektor-Erweiterungen, daher konzentriert sich diese Arbeit auf die skalaren Instruktionen.

Das Ziel dieser Erweiterungen ist es, dedizierte Instruktionen bereitzustellen, die die Performanz steigern und gleichzeitig den Speicherbedarf sowie das Risiko von Timing-Attacken reduzieren.

Die skalaren Kryptografie-Erweiterungen für RISC-V sind granular modular aufgebaut. So kann je nach Anwendungsfall nur Entschlüsselung mit einem bestimmten Algorithmus implementiert werden, ohne die gesamte Bandbreite an Instruktionen zu benötigen. Momentan sind folgende Erweiterungen spezifiziert:

- Zkne, Zknd: Erweiterungen für die AES-Verschlüsselung und -Entschlüsselung
- Zknh: Erweiterungen für die SHA2-Hashfunktionen
- Zksh, Zksed: ShangMi Hash- und Verschlüsselungs-Erweiterungen (Chinesischer Standard)
- Zkr: CSR-Register für eine Entropie-Quelle

Diese Extensionen sind unterschiedlich für RV32 und RV64 spezifiziert, da bestimmte Algorithmen auf 64-Bit-Architekturen effizienter implementiert werden können.

## 4.3 Advanced Encryption Standard

Der Advanced Encryption Standard (AES) ist ein symmetrischer Blockverschlüsselungs-Algorithmus, der vom NIST als Standard (FIPS 197) [9] festgelegt wurde. AES arbeitet auf einem Zustandsblock von 128 Bit, der intern als  $4 \times 4$  Matrix von Bytes organisiert ist.

Der Algorithmus ist rundenbasiert aufgebaut. Die Anzahl der Runden (10, 12 oder 14) hängt dabei von der gewählten Schlüssellänge (128, 192 oder 256 Bit) ab. Jede Standardrunde kombiniert vier fundamentale mathematische Operationen, um die kryptografischen Prinzipien der **Konfusion** und **Diffusion** sicherzustellen [10]:

- **SubBytes**: Eine nichtlineare Substitution jedes Bytes durch einen Wert aus einer Substitution-Box (S-Box).
- **ShiftRows**: Eine zyklische Verschiebung der Zeilen der Zustandsmatrix.
- **MixColumns**: Eine lineare Transformation, die die Spalten der Matrix vermischt (entfällt in der finalen Runde).
- **AddRoundKey**: Eine bitweise XOR-Verknüpfung des aktuellen Zustands mit dem Runden-schlüssel.

Die Entschlüsselung erfolgt durch die Anwendung der jeweiligen inversen Operationen in umgekehrter Reihenfolge.

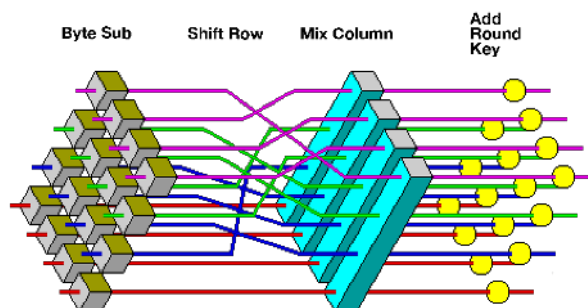


Abbildung 8: Struktur einer AES-Verschlüsselungsrunde

## 4.4 AES-Instruktionen für RV32

Die Erweiterungen Zkne und Zknd implementieren spezifische Instruktionen für die RV32, um die oben beschriebenen AES-Operationen effizient in Hardware abzubilden. Da der AES-Zustand 128 Bit umfasst, ein RV32-Register jedoch nur 32 Bit breit ist, operieren diese Instruktionen auf Teilaspekten des Zustands.

Es werden vier Instruktionen zur Verfügung gestellt, die zwischen Standardrunden („middle rounds“) und der Finalrunde unterscheiden:

- **aes32esmi** (Middle-round encryption): Führt SubBytes und MixColumns aus.

- `aes32esi` (Final-round encryption): Führt nur die SubBytes-Operation aus (da MixColumns in der letzten Runde entfällt).
- `aes32dsmi` (Middle-round decryption): Inverses Gegenstück zur Verschlüsselung.
- `aes32dsi` (Final-round decryption): Inverse Finalrunde.

#### 4.4.1 Funktionsweise

Alle vier Instruktionen nutzen das gleiche Format ([Abbildung 9](#)) mit drei Operanden und einem Byte-Selektor: `rd` (output), `rs1` (key), `rs2` (state), `bs` (byte selector).

1. Aus dem Quellregister `rs2` wird basierend auf dem 2-Bit Selektor `bs` ein einzelnes Byte (8 Bit) ausgewählt.
2. Auf dieses Byte wird die S-Box (Vorwärts oder Invers) angewendet. Bei den `smi`-Varianten folgt zusätzlich eine (partielle) MixColumn-Transformation.
3. Das Ergebnis wird an die Position geschoben, die durch `bs` definiert ist, und per XOR mit dem Inhalt von `rs1` verknüpft. Das Resultat wird in `rd` gespeichert.

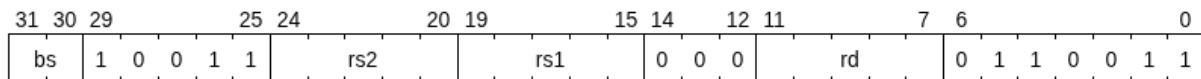


Abbildung 9: AES-Instruktionsformat

#### 4.4.2 Implementierungsaspekte im PicoNut

Nachfolgend werden zentrale Aspekte der Implementierung der AES-Instruktionen im PicoNut-Prozessor sowie deren Integration erläutert.

**scalar\_crypto-Modul** Die Implementierung der AES-Instruktionen im PicoNut-Projekt wurde bewusst einfach gehalten, wobei die funktionale Korrektheit zunächst oberste Priorität hatte. Die Umsetzung orientiert sich eng an der Referenzimplementierung in Sail [\[11\]](#).

Die S-Box wurde als Lookup-Tabelle (LUT) innerhalb der FPGA-Ressourcen realisiert, während die übrigen Operationen direkt in C++ umgesetzt wurden. Der Ressourcenbedarf hierfür beträgt etwa 800 LUTs und 10 Flip-Flops.

Eine Besonderheit stellt die MixColumn-Operation dar. Da diese Berechnungen im Galois-Feld  $GF(2^8)$  (bzw.  $GF(256)$ ) erfolgen, mussten spezifische Hilfsfunktionen für die Addition und Multiplikation innerhalb dieses Körpers implementiert werden.

In der Mathematik ermöglichen Galois-Felder die Anwendung von Körperarithmetik auf Mengen mit nicht-primer Ordnung. In der symmetrischen Kryptografie sind diese Körper essenziell, um mathematisch strukturierte und gleichzeitig umkehrbare Operationen auf Byte-Ebene zu realisieren.

In [Listing 4](#) ist die Implementierung der Multiplikation mit 2 in  $GF(256)$  in C++ sowie die zugrunde liegende mathematische Definition dargestellt (wobei  $a_n$  die Bit-Werte repräsentieren). Entscheidend ist hierbei, dass das Ergebnis jeder Multiplikation modulo des irreduziblen Polynoms  $x^8 + x^4 + x^3 + x + 1$  reduziert wird, um innerhalb der Feldgrenzen zu bleiben.

$$A(x) = a_7x^7 + a_6x^6 + a_5x^5 + a_4x^4 + a_3x^3 + a_2x^2 + a_1x + a_0$$

$$B(x) = (x * A(x) \bmod x^8 + x^4 + x^3 + x + 1)$$

```
static sc_uint<8> gf256_2_mul(sc_uint<8> A) {
    sc_uint<8> B = A << 1;
    if(A[7] == 1) // B will have x^8 term
    {
        B ^= 0x1B;
    }
    return B;
}
```

Listing 4: Galois-Feld Multiplikation mit 2 in GF(256)

In der ersten Iteration wurden alle AES-Operationen innerhalb eines einzigen Taktzyklus ausgeführt. Dies führte jedoch zu erheblichen Timing-Einbußen – konkret zu einer Reduktion der Taktfrequenz von 43 MHz auf 34 MHz in der FPGA-Synthese. Daher wurde die Implementierung angepasst, um die Berechnungen über zwei Taktzyklen zu verteilen. Diese Änderung stabilisierte das Timing des Prozessors, ohne die Gesamtleistung negativ zu beeinflussen.

**Integration in den Prozessor** Ein Modul wie `scalar_crypto` war im Referenz-Nucleus zu Beginn der Arbeit nicht vorgesehen. Da jedoch im Wintersemester 2024/25 parallel die M-Extension entwickelt wurde, standen ähnliche Anforderungen für Rechenwerke mit mehreren Taktzyklen im Raum. Zunächst wurden zwei verschiedene Ansätze für die Integration solcher Multi-Zyklus-Operationen verfolgt.

Die Timing-Analyse zeigte schließlich, dass die Implementierung von Tristan Kundrat für die M-Extension den kritischen Pfad weniger belastete. Infolgedessen wurden die AES-Instruktionen auf Basis dieser Implementierung neu aufgesetzt.

Die Lösung von Tristan Kundrat nutzt dedizierte `start`- und `valid`-Signale zur Steuerung des Moduls. Dadurch kann der Nucleus-Controller das Modul über die Arithmetic Logic Unit (ALU) asynchron ansteuern und effizient auf die Fertigstellung der Berechnung warten.

## 4.5 Software

Die RV32 AES-Instruktionen sind bewusst minimalistisch gestaltet, um eine ressourceneffiziente Hardware-Implementierung zu ermöglichen. Daher müssen diese Instruktionen in Software kombiniert werden, um eine vollständige AES-Verschlüsselung oder -Entschlüsselung zu realisieren.

Basierend auf der Beispielimplementierung der RISC-V Cryptography Task Group [11] wurde eine Softwarebibliothek entwickelt, welche die AES-Instruktionen nutzt, um den gesamten kryptografischen Prozess abzubilden. Diese Bibliothek umfasst Funktionen für die Schlüssel-erweiterung (Key Expansion) sowie für die Ver- und Entschlüsselung von Datenblöcken im ECB-Modus (Electronic Codebook) und CTR-Modus (Counter Mode).

Der ECB-Modus ist der einfachste Betriebsmodus für Blockchiffren, bei dem jeder Datenblock unabhängig verschlüsselt wird. Dies führt jedoch dazu, dass identische Klartextblöcke zu identischen Geheimtextblöcken führen, was den Modus anfällig für Mustererkennung macht.

Im Gegensatz dazu wandelt der CTR-Modus eine Blockchiffre in eine Stromchiffre um, indem ein Zählerwert verschlüsselt und das Ergebnis mit dem Klartext XOR-verknüpft wird. Dies bietet eine deutlich höhere Sicherheit, da identische Klartextblöcke unterschiedliche Chiffre erzeugen. Zudem ermöglicht der CTR-Modus die effiziente Verarbeitung von Datenströmen variabler Länge und lässt sich leicht parallelisieren.

## 4.6 Ergebnisse

Abschließend werden die Leistungsanalyse der AES-Erweiterungen im PicoNut-Projekt sowie ein Demonstrator zur Veranschaulichung der Verschlüsselung vorgestellt.

### 4.6.1 Analyse

Die Implementierung der skalaren AES-Erweiterung im PicoNut-Projekt ermöglicht eine signifikante Leistungssteigerung im Vergleich zu rein softwarebasierten Ansätzen auf Standard-RISC-V-Kernen.

Ein Vergleich zwischen der TinyAES-Softwarebibliothek [12] (optimiert mit -O3) und der PicoNut-Implementierung unter Verwendung der AES-Instruktionen ergab einen Speedup von etwa  $6 \times$  bei der Ver- und Entschlüsselung im ECB-Modus.

Ein weiterer Vorteil dieser Implementierung liegt in der etwas höheren Sicherheit gegenüber Seitenkanalangriffen, die Schlüssel (Keys) beispielsweise durch Stromprofile extrahieren können. Durch die Realisierung der S-Boxen als LUTs in der FPGA-Hardware wird weniger Energie benötigt, da die S-Box nicht über den Systembus geladen werden muss – was eines der ersten Ziele der akademischen Differential Power Analysis war [13]. Zudem ist das Zeitverhalten dieser AES-Implementierung konstant. Jedoch sind diese Verbesserungen nur begrenzt wirksam, da für eine wirklich sichere Implementierung ein weitaus größerer Test- und Absicherungsaufwand betrieben werden müsste.

### 4.6.2 Demonstrator

Die Demonstration von Kryptografie ist oft abstrakt, da sie primär den Unterschied zwischen lesbaren und unlesbaren Daten zeigt. Um die Auswirkungen der verschiedenen Modi greifbar zu machen, wurde ein Demonstrator mit Videoausgabe entwickelt. Dieser zeigt live, wie ein Videostream verschlüsselt und entschlüsselt wird. In [Abbildung 10](#) ist ein Screenshot dieses Demonstrators dargestellt, der zur Dokumentation mittels der SDL2-Bibliothek für x86\_64 Linux nachgebildet wurde.

Am Bild lässt sich der Unterschied zwischen dem unverschlüsselten Original und den verschlüsselten Streams deutlich erkennen. Besonders hervorgehoben wird die Schwäche des ECB-Modus durch deutlich sichtbare Muster im Chiffre, während der CTR-Modus bei gleichem Schlüssel keine Strukturen im Output hinterlässt. Dies verdeutlicht den Studierenden, dass

die Sicherheit eines Systems nicht allein von der Stärke des Algorithmus, sondern maßgeblich von der korrekten Wahl des Betriebsmodus abhängt.

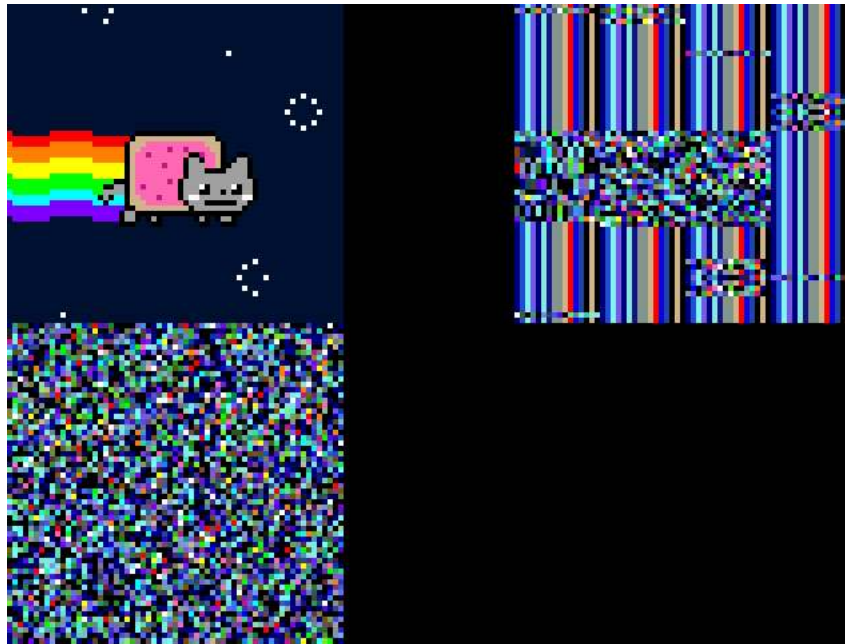


Abbildung 10: Demonstrator der AES-Verschlüsselung (nachgebaut mit SDL2)

## 5 RISCOF

RISCOF (**RISC-V Compliance Framework**) ist ein Framework zur Überprüfung der Konformität von RISC-V-Implementierungen mit dem offiziellen RISC-V-Standard. Dabei wird die Implementierung in einer Reihe von Tests gegen eine Referenz geprüft. Es wurde von InCore Semiconductors entwickelt [14].

Durch die Erweiterung des PicoNut-Prozessors um die in dieser Arbeit behandelten Extensions ist es notwendig, die Konformität der neuen Implementierung sicherzustellen. Gleichzeitig muss die korrekte Funktion des bestehenden Basis-Instruktionssatzes kontinuierlich validiert werden. Daher ist RISCOF ein zentraler Bestandteil dieser Projektarbeit.

### 5.1 Funktionsweise

Die Funktionsweise und Konfiguration von RISCOF wird im Folgenden erläutert.

#### 5.1.1 Überblick

Die Funktionsweise von RISCOF ist in [Abbildung 11](#) schematisch dargestellt. RISCOF nutzt ein Plugin-System, bei dem jedes Plugin definiert, wie Tests für eine spezifische RISC-V-Implementierung – das Device Under Test (DUT), z. B. PicoNut – erstellt und ausgeführt werden. Für jeden Testlauf wird derselbe Code sowohl auf dem DUT als auch auf der Referenzimplementierung, dem **RISC-V Sail Model**, ausgeführt. Letzteres dient dabei als „Golden Reference“.

Jeder ausgeführte Test generiert eine Signaturdatei, die einen Snapshot eines definierten Speicherbereichs enthält. Diese Signaturen werden anschließend zwischen dem DUT und dem Referenzmodell verglichen, um Abweichungen festzustellen.

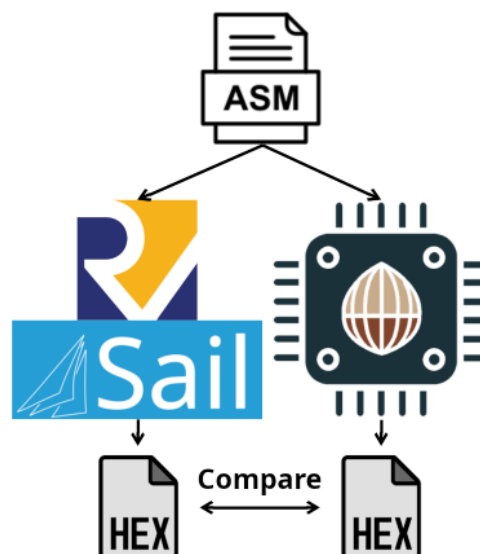


Abbildung 11: Schematische Funktionsweise von RISCOF

Die eigentlichen Testfälle stammen aus dem **riscv-arch-test**-Repository der RISC-V-Community und zielen auf spezifische Teile des Befehlssatzes ab.

## 5.1.2 Konfiguration

Die Auswahl der ausgeführten Tests erfolgt über die Definition der Architektur in der Datei `piconut_isa.yaml`. Durch das Bearbeiten dieser Datei können spezifische RISC-V-Erweiterungen (Extensions) aktiviert oder deaktiviert werden, indem die Felder ISA und MISA entsprechend konfiguriert werden. Die erforderlichen Bit-Einstellungen sind im Abschnitt „3.1.1 Machine ISA Register (misa“ der privilegierten RISC-V-Spezifikation festgelegt.

## 5.2 Integration

Die Einbindung von RISCOF erfordert die Implementierung eines Plugins in Python als Schnittstelle zwischen RISCOF und dem PicoNut-Prozessor.

### 5.2.1 RISCOF-PicoNut Plugin

Die Implementation des PicoNut-RISCOF-Plugins basiert auf dem offiziellen Sail-Plugin. Dieses Plugin definiert die Schritte zum Kompilieren und Ausführen der Tests auf dem PicoNut-Core.

Darüber hinaus wird die Testumgebung über einen C-Header an die Eigenheiten des Prozessors angepasst. Dies betrifft beispielsweise den Halt-Befehl (beim PicoNut aktuell via `ecall`).

Von zentraler Bedeutung ist zudem die Definition, wie die Signaturdaten an das Host-System übertragen werden. Üblicherweise wird hierfür eine UART-Schnittstelle genutzt, um die Speicherdaten seriell zu senden. Da zum Zeitpunkt der Implementierung die Erfahrung mit der UART-Schnittstelle des PicoNuts fehlte und die Tests primär in der Simulation durchgeführt wurden, wurde stattdessen ein Mechanismus implementiert, der den Speicherinhalt direkt aus dem Simulator ausliest und in einer Datei speichert.

Für Tests auf physischer Hardware oder die Nutzung von RISCOF für andere PicoNut-Varianten wird eine UART-Anbindung erforderlich sein; deren Implementierung ist jedoch nicht Bestandteil dieser Arbeit.

### 5.2.2 Eigene Tests (Custom Tests)

Obwohl RISCOF Tests standardmäßig aus dem offiziellen `riscv-arch-test`-Repository bezieht, ermöglicht das PicoNut-Projekt das Hinzufügen benutzerdefinierter Tests. Diese können im Verzeichnis `sw/verification/riscov/custom-tests` abgelegt werden.

Vor der Ausführung von RISCOF werden diese Testfälle temporär in das lokale `riscv-arch-test`-Repository kopiert. Dies erlaubt eine spezifische Testabdeckung und gezielte Regressionstests für den PicoNut.

## 5.3 Ergebnisse

Die erfolgreiche Integration von RISCOF in das PicoNut-Projekt ermöglicht eine kontinuierliche Validierung der RISC-V-Konformität während der Entwicklung. Für die Implementierung

der A-, M- und Zkne/d-Extensions konnte so sichergestellt werden, dass keine Regressionen im Basis-Instruktionssatz auftreten und die neuen Funktionalitäten korrekt umgesetzt sind.

Dennoch weist RISCOF Schwachstellen auf. Neben der mäßigen Benutzerfreundlichkeit des Command-Line-Interface (CLI) gestaltet sich insbesondere die Fehlersuche bei Testfehlschlägen schwierig, da lediglich die End-Signaturen der Ergebnisse verglichen werden. Da einzelne Tests unterschiedlich viele Werte in die Signatur schreiben (teils nur einen 32-Bit-Wert, teils mehrere), ist es oft mühsam, allein anhand der Zeilennummer in der Signaturdatei auf den exakten fehlerhaften Testschritt rückzuschließen.

## 5.4 Ausblick

Seit 2025 wird in der RISC-V-Community an einer Ablösung von RISCOF gearbeitet. Eine inoffizielle Dokumentation dieser neuen Struktur wurde am 3. Januar 2026 im `riscv-arch-test`-Repository veröffentlicht [\[15\]](#). Die neue Version setzt im Gegensatz zu RISCOF auf selbstvalidierende Tests, die direkt auf den Ergebnissen von Sail basieren. Dies vereinfacht das Setup erheblich, da einzelne Testfehlschläge präziser identifiziert werden können.

Da diese neue Version jedoch noch nicht finalisiert wurde, bleibt RISCOF für das PicoNut-Projekt vorerst weiterhin maßgeblich. Zukünftige Arbeiten könnten sich mit der Integration des Nachfolgers befassen, sobald dieser einen stabilen Zustand erreicht hat.

## 6 Nebenaufgaben

---

Das PicoNut-Projekt ist über das letzte Jahr stetig gewachsen, was die auch viel Bedarf an Restrukturierung und Verbesserung der Infrastruktur mit sich brachte. Darüber hinaus enden manche Projekte nicht mit Fertigstellung, sondern durch zeitliche Beschränkungen. Dies motiviert neben den Hauptprojekten auch Nebenaufgaben im PicoNut-Projekt zu verfolgen. Einige davon werden hier beleuchtet.

### 6.1 ASCII-Tetris

Das PicoNut-Projekt wird jedes Halbjahr auf dem Projekttag der Fakultät Informatik der Technischen Hochschule Augsburg präsentiert. Zum Projekttag im Sommersemester 2025 waren bereits alle notwendigen Komponenten für ein funktionsfähiges Spiel fertiggestellt, jedoch existierte noch keine Anwendung, die diese Teilsysteme in einer gemeinsamen Präsentation vereinte. Um diese Lücke zu schließen und die Fähigkeit des Prozessors anschaulich zu demonstrieren, wurde ein ASCII-Tetris-Spiel für den PicoNut portiert.

#### 6.1.1 Softwareauswahl

Bei der Suche nach einer geeigneten Implementierung im Internet stand die Kompatibilität im Vordergrund. Die Wahl fiel auf das Projekt [Gregwar/ASCII-Tetris \[12\]](#), da es in der Programmiersprache C geschrieben ist, eine einfache ASCII-Ausgabe nutzt und unter einer passenden Lizenz vorliegt.

#### 6.1.2 Portierung

Die eigentliche Portierung erforderte gezielte Anpassungen am Quellcode, insbesondere bei der Implementierung der Sleep-Funktionen und der Synchronisation der Spielgeschwindigkeit.

**Steuerung** Da der PicoNut zum Zeitpunkt der Portierung noch über keine GPIO-Anbindung für physische Knöpfe verfügte, musste die Spielsteuerung über das UART-Interface realisiert werden. Hierfür wurde das Register genutzt, welches den zuletzt empfangenen ASCII-Wert speichert, sodass das Spiel bequem über eine serielle Konsole gesteuert werden kann. Da der PicoNut-Simulator ursprünglich keine UART-Eingaben unterstützte, wurde diese Funktionalität im Zuge der Entwicklung nachgerüstet, um das Testen der Spielelogik auch ohne FPGA-Board zu ermöglichen.

**Melodie** Ein besonderes Highlight der Präsentation war die Integration der berühmten Tetris-Melodie. Diese wurde über ein von Tristan Kundrat entwickeltes Synthesizer-Audio-Modul ausgegeben, das bereits auf dem PicoNut lauffähig war.

Herausforderungen ergaben sich jedoch durch die Weiterentwicklung des Prozessors: Nach dem Mergen der Interrupt-Unterstützung und der A-Extension funktionierte das ursprüngliche Busy-Waiting in Software nicht mehr präzise, wodurch die Melodie aus dem Takt geriet. Da der PicoNut als Single-Core-System die Spielelogik und die Audio-Ausgabe nicht einfach

parallel verarbeiten konnte, wurde der neu implementierte Interrupt-Support genutzt. Durch das Triggern der Audio-Ausgabe in festen Zeitintervallen konnte die Melodie abgespielt werden und ein flüssiger Spielablauf erreicht werden.

In [Abbildung 12](#) ist ein Screenshot des laufenden Spiels auf dem PicoNut zu sehen.



Abbildung 12: ASCII-Tetris auf dem PicoNut-Prozessor über das UART-Interface

## 6.2 GitLab CI Migration

Ein zentraler Grundsatz des PicoNut-Projekts war die parallele Entwicklung der Continuous Integration (CI) Infrastruktur. Zu Projektbeginn wurde hierfür ein Jenkins gearbeitet, welches die Kompilierung, Synthese und Verifikation des PicoNut-Codes übernahm. Im Projektverlauf identifizierte Probleme führten jedoch zu der Entscheidung, das System auf GitLab CI zu migrieren.

### 6.2.1 Probleme mit Jenkins

Die Analyse der aktuellen Jenkins-Konfiguration verdeutlichte verschiedene Defizite, die den Entwicklungsprozess beeinträchtigten:

- **Redundante Skripte:** Durch das projektübergreifende Kopieren von Build-Skripten entstanden zahlreiche Redundanzen. Dies erschwerte eine konsistente Wartung und führte zu einem hohen Synchronisationsaufwand bei globalen Änderungen.
- **Mangelnde Integration:** Die CI-Runs erfolgten primär manuell oder „nightly“. Dies verhin- derte eine frühzeitige Validierung von Änderungen vor dem Merge in den Hauptzweig. Da die CI-Skripte außerhalb des Hauptrepositories gelagert waren, entkoppelte sich deren Entwick- lung vom Quellcode, was zu Inkonsistenzen führte.
- **E-Mail-Spam:** Fehlermeldungen der Nightly-Runs wurden an das gesamte Team versendet, was zu einer Überflutung der Postfächer führte.

### 6.2.2 Migration

Der Umbau des PicoNut-Build-Systems war der entscheidende Schritt, um die Migration überhaupt erst sinnvoll machbar zu machen. Indem wir alle Testabläufe in ein zentrales Make-Target zusammengeführt haben, ließ sich die Konfiguration für die CI (Continuous Integration) wesentlich einfacher und übersichtlicher gestalten.

Die Umsetzung lässt sich in drei Bereiche unterteilen:

**Runner-Infrastruktur** Die GitLab Runner direkt auf dem bisherigen Jenkins-Server installiert. Damit Entwickler nicht in einer Warteschlange hängen, wurden mehrere Runner registriert. Über den `concurrent`-Wert in der `config.toml` haben wir eingestellt, dass mehrere Jobs gleichzeitig laufen können, was die Wartezeit bei vielen Änderungen deutlich verkürzt [16].

**Umgang mit Docker-Images** Das Docker-Image für PicoNut ist wegen der enthaltenen Toolchains mit etwa 10 GB sehr groß. Würde GitLab dieses Image bei jedem Durchlauf neu herunterladen, würde das die Pipeline extrem ausbremsen. Daher haben wir die `pull_policy` auf `never` gesetzt. Das Image liegt nun fest auf dem Server bereit. Für Updates wurde interne Dokumentation angelegt.

**Aufbau der Pipeline** Die gesamte Steuerung läuft über eine `.gitlab-ci.yml` Datei. Wir haben den Prozess in zwei klare Abschnitte unterteilt:

- Build-Phase: Hier werden die Module kompiliert, die FPGA-Bitfiles erstellt und die Dokumentation erzeugt.
- Test-Phase: Hier laufen die Modul- und Systemtests

### 6.2.3 Verbesserungen

Die Umstellung auf GitLab CI hat den Entwicklungsprozess nicht besser integriert, sondern auch um Funktionen erweitert.

**Automatisierte Qualitätssicherung** Durch die direkte Verknüpfung mit dem Git-Workflow startet nun bei jedem Push sowie jedem Merge-Request sofort eine automatisierte Pipeline. Branch-Schutz stellt sicher, dass Code nur dann in den `develop`-Branch einfließt, wenn er sämtliche Build- und Teststufen erfolgreich bestanden hat. Da die Docker-Umgebung zudem eine konsistente Basis garantiert, ist das System jederzeit in einem validierten Zustand, wodurch zeitgesteuerte Nightly-Builds vollständig entfallen. Nightly-Builds sind auch in GitLab als Scheduled Pipelines verfügbar.

**Dokumentations-Checks** Ergänzend zu den funktionalen Tests wurde ein Check für die Sphinx-Dokumentation integriert. Es wird nun geprüft, ob die Dokumentation ohne Warnungen kompiliert wurde. Dies stellt sicher, dass alle Quellverweise und Referenzen aktuell sind.

**Vorschau der Dokumentation** Ein neues Feature für das Peer-Review ist die automatische Bereitstellung der Dokumentation. Sobald ein Merge-Request erstellt wird, generiert die Pipeline den aktuellen Stand und lädt ihn auf einen internen Webserver hoch. Über die „Deployment“-Funktion von GitLab erhalten Reviewer direkt im MR einen Link („View App“), um die Dokumentation ohne lokalen Build-Aufwand zu prüfen [17]. Nach Abschluss oder Ablehnung des Merge-Requests bereinigt die Pipeline die Dokumentation auf dem Server selbstständig.

## 6.3 Video-Interface

Parallel zur Infrastruktur-Modernisierung wurde die Hardware-Anbindung des Video-Moduls vorangetrieben. Ursprünglich im Sommersemester 2025 als eigenständiges VGA-Modul von

Martin Erichsen und Bearel I. Ngaleu [18], [19] entwickelt, erzeugte das Modul zwar ein Testbild in FPGA-Logik, verfügte jedoch über keine Anbindung an den PicoNut-Prozessor. Um die Präsentation des PicoNut-Projekts auf dem Projekttag im Wintersemester 2025/26 aufzuwerten, wurde die Integration des Video-Interfaces als Nebenaufgabe verfolgt.

### 6.3.1 Identifizierte Defizite

Vor der Integration wurden verschiedene Schwachstellen analysiert, die den produktiven Einsatz des Moduls verhinderten:

- So fehlte die Datei `line_column_counter.cpp`, die für die aktuellen Pixel-Position verantwortlich ist.
- Die existierende Dokumentation zur Register-Schnittstelle sollte eigentlich kompatibel mit einer reinen Simulationsvariante des Moduls sein, jedoch stimmten die Register-Adressen nicht mit dem Quellcode überein.
- Farbpaletten (Color Maps), die auf dem begrenzten Speicherplatz des Framebuffers mit weniger bits pro Pixel arbeiten, wurden nicht unterstützt.
- Es wurde kein vollständiges Beispielsystem bereitgestellt, das die Nutzung des Moduls demonstriert. Dadurch gab es keine Constraints-Datei für die Pinbelegung auf dem FPGA-Board und das Signal konnte nicht verglichen werden.

### 6.3.2 Wishbone Anbindung

Die Integration des Video-Moduls in den PicoNut-Prozessor erfolgte über eine Wishbone-Schnittstelle.

Hierfür wurde ein neues Slave-Modul `video_wb` erstellt, das die Register des Video-Moduls über den Wishbone-Bus ansprechbar macht.

Die Register-Adressen wurden dabei so gewählt, dass alle nötigen Register für die schon existierende Software für das Modul `c_soft_graphics` abgedeckt sind. So wurden Register für Höhe und Breite des Framebuffers in das Hardware Modul übernommen, da in Hardware die Framebuffergröße stark eingeschränkt ist. Es wurden jedoch auch ein paar Anpassungen an der Ordnung bestehender Register vorgenommen, um eine einfache Struktur zu gewährleisten.

Für die von beiden Implementierungen nicht implementierte Register für eine Color Map wurde in der Wishbone-Schnittstelle readonly Register angelegt.

Um den Framebuffer 32-bit breit anzusprechen, jedoch nur 1 Byte pro Pixel zu nutzen, wurde die Wishbone-Schnittstelle so implementiert, dass bei der Adresse die letzten 2 Bits ignoriert werden. So kann später auch ein 16-bit oder 24-bit Framebuffer unterstützt werden, ohne die Adressierung zu ändern.

Bei der Implementierung des Moduls konnte auf bestehende Implementierungen im PicoNut-Projekt aufgebaut werden, wie z.B. die I2C-Wishbone Schnittstelle.

### 6.3.3 Funktionserweiterungen

Neben der reinen Integration wurden zwei wesentliche Features implementiert, um die visuelle Qualität und Flexibilität zu erhöhen.

**Farbpaletten-Unterstützung (Color Maps)** Um den begrenzten Block-RAM (BRAM) des FPGAs effizient zu nutzen, wurde eine Unterstützung für Farbtabelle integriert. Da RGB-Werte mit lediglich 8 Bit (RGB332) visuell unzureichend sind ([Abbildung 13](#)), erlauben die Paletten eine Indizierung hochwertigerer Farben bei geringem Speicherbedarf pro Pixel.

Es wurden vier Paletten (4, 8, 16 und 256 Farben) in SystemC implementiert. Der Ressourcenverbrauch stieg dadurch moderat um ca. 1300 LUTs an, was in Anbetracht der verbesserten Bildqualität als vertretbar eingestuft wurde.

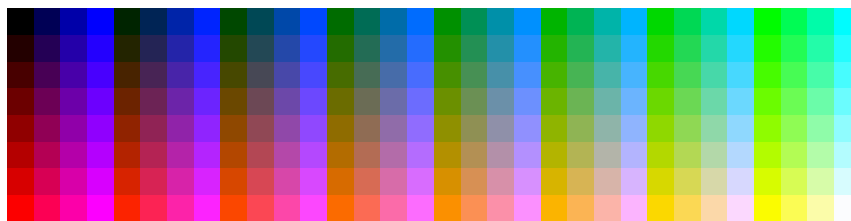


Abbildung 13: 256 Farben der RGB332 Farbpalette

**Konfigurierbares Scaling** Das Video-Modul gibt standardmäßig ein 640x480 VGA-Signal aus, weil dies technisch einfach über die typische FPGA-Taktfrequenz von 25 MHz realisierbar ist. Momentan wird für den Framebuffer der internen Block-RAM des FPGA genutzt, das einzige Board, das im Projekt verwendet wird, das ULX3S mit Lattice ECP5 hat jedoch nur 400 KB Block-RAM (3.66 Mbit, effektiv nutzbar: 0x68000 Bytes) [20], was für einen 640x480 Framebuffer mit 8 Bit pro Pixel (307200 Bytes) gerade so ausreicht, jedoch nicht viel Platz für Software lässt, weil auch der RAM des Prozessors im selben Block-RAM liegt.

Daher war schon von Martin Erichsen Nearest-Neighbor Scaling im Video-Modul implementiert worden, mit einem hardcodiertem Skalierungsfaktor von 16. Das waren jedoch zu wenige Pixel um erkennbare Bilder darzustellen.

Anstelle eines festen Skalierungsfaktors kann dieser nun über eine statische Konfigurationsvariable definiert werden. In [Listing 5](#) ist die Implementierung dargestellt. Da die Divisionen und Multiplikationen durch konstante Faktoren erfolgen, konnte dies ressourcensparend über Shift-Adder-Operationen realisiert werden.

```
column = vid_column_in.read() /  
        ((uint32_t)PN_CFG_VIDEO_FB_SCALING_DIVISOR);  
line   = vid_line_in.read() /  
        ((uint32_t)PN_CFG_VIDEO_FB_SCALING_DIVISOR);  
ram_addrb = column +  
            line * (640 / ((uint32_t)PN_CFG_VIDEO_FB_SCALING_DIVISOR));
```

Listing 5: Implementierung des konfigurierbaren Scalings

## 6.4 Weitere Tätigkeiten

Neben den bisher beschriebenen Nebenaufgaben wurden von mir im Projektverlauf weitere Aufgaben übernommen:

- Experimente mit FreeRTOS auf dem PicoNut-Prozessor [Issue #21](#)
- Migration von Clint, Timer, Audio-System, ELF2Mem und Doxygen für ein neu eingeführtes Build-System
- Peer-Review von Soft- und Hardware-Modulen für Audio, Video und Clint

## 7 Fazit

---

Das abschließende Kapitel fasst die Ergebnisse der Arbeit zusammen und gibt einen Ausblick auf mögliche zukünftige Entwicklungen.

### 7.1 Zusammenfassung

Im Rahmen dieser Projektarbeit wurde der PicoNut-Prozessor erfolgreich um wesentliche Befehlssatz-Erweiterungen ergänzt und die Entwicklungsinfrastruktur modernisiert. Durch die Implementierung der A-Extension wurde die hardwareseitige Basis für die Synchronisation in Multi-Core-Systemen und die Ausführung moderner Betriebssysteme wie Linux geschaffen. Die Integration der skalaren Kryptografie-Erweiterungen (Zkne/Zknd) ermöglichte zudem eine signifikante Beschleunigung von AES-Verschlüsselungen um den Faktor 6, was die Effizienz des Prozessors in sicherheitskritischen Anwendungen demonstriert.

Darüber hinaus wurde durch die Migration auf GitLab CI und die Integration des RISCOF-Frameworks die Qualitätssicherung professionalisiert. Die Anbindung eines Video-Interfaces inklusive konfigurierbarem Scaling und Farbpaletten-Support erweitert die Vielseitigkeit des Systems und ermöglichte anschauliche Demonstratoren.

### 7.2 Ausblick

Trotz der erreichten Meilensteine bleibt Raum für zukünftige Optimierungen. Die implementierte A-Extension entfaltet ihr volles Potenzial erst mit der geplanten Erweiterung des PicoNut zu einem Multi-Core-System, bei dem die entwickelten Reservierungsmechanismen in der Membrana die Datenkonsistenz zwischen verschiedenen Kernen sicherstellen müssen.

Ein weiterer wichtiger Schritt ist die fortlaufende Anpassung an die sich entwickelnden RISC-V-Standards. Die absehbare Ablösung von RISCOF durch selbstvalidierende Architekturtests bietet die Chance, die Verifikation noch präziser und wartungsfreundlicher zu gestalten. Zudem könnte die Kryptografie-Einheit in Zukunft um weitere Algorithmen wie SHA2 (Zknh) oder durch die Implementierung von Vektor-Instruktionen erweitert werden, um den PicoNut als vielseitige Plattform für die Forschung an sicheren eingebetteten Systemen weiter zu etablieren.

# I Abbildungsverzeichnis

---

|              |                                                                                         |    |
|--------------|-----------------------------------------------------------------------------------------|----|
| Abbildung 1  | Aufbau eines PicoNut-Prozessorkerns .....                                               | 5  |
| Abbildung 2  | Codierung der AMO-Instruktionen .....                                                   | 9  |
| Abbildung 3  | Nucleus ohne A-Extension .....                                                          | 11 |
| Abbildung 4  | Nucleus mit A-Extension .....                                                           | 11 |
| Abbildung 5  | A-Extension Controller States (grün für Eingangssignale, rot für Ausgangssignale) ..... | 12 |
| Abbildung 6  | Ablauf einer AMO-Instruktion im PicoNut-Prozessor .....                                 | 12 |
| Abbildung 7  | Codierung der Ir/sc-Instruktionen .....                                                 | 13 |
| Abbildung 8  | Struktur einer AES-Verschlüsselungsrunde .....                                          | 19 |
| Abbildung 9  | AES-Instruktionsformat .....                                                            | 20 |
| Abbildung 10 | Demonstrator der AES-Verschlüsselung (nachgebaut mit SDL2) ....                         | 23 |
| Abbildung 11 | Schematische Funktionsweise von RISCOF .....                                            | 24 |
| Abbildung 12 | ASCII-Tetris auf dem PicoNut-Prozessor über das UART-Interface ..                       | 28 |
| Abbildung 13 | 256 Farben der RGB332 Farbpalette .....                                                 | 31 |

## II Quellcodeverzeichnis

---

|           |                                                                                                           |    |
|-----------|-----------------------------------------------------------------------------------------------------------|----|
| Listing 1 | Beispiel für die Verwendung von <code>stdatomic.h</code> zur Implementierung eines atomaren Zählers ..... | 9  |
| Listing 2 | Beispiel für die Verwendung von AMOs zur Implementierung eines Spinlocks (von <a href="#">[4]</a> ) ..... | 10 |
| Listing 3 | Implementierung einer Compare-and-Swap (CAS) Funktion mittels LR/SC .....                                 | 15 |
| Listing 4 | Galois-Feld Multiplikation mit 2 in $GF(256)$ .....                                                       | 21 |
| Listing 5 | Implementierung des konfigurierbaren Scalings .....                                                       | 31 |

# III Literaturverzeichnis

---

- [1] Intel Corporation, „intel/systemc-compiler: This tool translates synthesizable SystemC code to synthesizable SystemVerilog.“. Zugriffen: 24. Januar 2026. [Online]. Verfügbar unter: <https://github.com/intel/systemc-compiler>
- [2] B. W. Mezger, D. A. Santos, L. Dilillo, C. A. Zeferino, und D. R. Melo, „A Survey of the RISC-V Architecture Software Support“, *IEEE Access*, Bd. 10, Nr. , S. 51394–51411, 2022, doi: [10.1109/ACCESS.2022.3174125](https://doi.org/10.1109/ACCESS.2022.3174125) .
- [3] RISC-V International, „The RISC-V Instruction Set Manual Volume I - Unprivileged Architecture“, technical report, Mai 2025. Zugriffen: 10. Juli 2025. [Online]. Verfügbar unter: <https://lf-riscv.atlassian.net/wiki/spaces/HOME/pages/16154769/RISC-V+Technical+Specifications>
- [4] RISC-V International, „Chapter 13. "A" Extension for Atomic Instructions“. 12. Mai 2025. Zugriffen: 10. Juli 2025. [Online]. Verfügbar unter: <https://lf-riscv.atlassian.net/wiki/spaces/HOME/pages/16154769/RISC-V+Technical+Specifications>
- [5] The Linux Kernel Archives, „Linux Kernel Source Tree“. Zugriffen: 10. Juli 2025. [Online]. Verfügbar unter: <https://git.kernel.org/pub/scm/linux/kernel/git/torvalds/linux.git/>
- [6] Wikipedia contributors, „Consistency model - Wikipedia“. Zugriffen: 10. Oktober 2025. [Online]. Verfügbar unter: [https://en.wikipedia.org/wiki/Consistency\\_model](https://en.wikipedia.org/wiki/Consistency_model)
- [7] R. Rajwar und J. Goodman, „Speculative lock elision: enabling highly concurrent multithreaded execution“, in *Proceedings. 34th ACM/IEEE International Symposium on Microarchitecture. MICRO-34*, 2001, S. 294–305. doi: [10.1109/MICRO.2001.991127](https://doi.org/10.1109/MICRO.2001.991127) .
- [8] Bundesamt für Sicherheit in der Informationstechnik, „BSI - Cyber Resilience Act“. Zugriffen: 24. Januar 2026. [Online]. Verfügbar unter: [https://www.bsi.bund.de/EN/Themen/Unternehmen-und-Organisationen/Informationen-und-Empfehlungen/Cyber\\_Resilience\\_Act/cyber\\_resilience\\_act\\_node.html](https://www.bsi.bund.de/EN/Themen/Unternehmen-und-Organisationen/Informationen-und-Empfehlungen/Cyber_Resilience_Act/cyber_resilience_act_node.html)
- [9] National Institute of Standards and Technology, „ADVANCED ENCRYPTION STANDARD (AES)“. Zugriffen: 24. Januar 2026. [Online]. Verfügbar unter: <https://nvlpubs.nist.gov/nistpubs/fips/nist.fips.197.pdf>
- [10] B. Schneier, *Applied Cryptography*, 2nd Aufl. Wiley, 1996. [Online]. Verfügbar unter: [http://www.amazon.com/Applied-Cryptography-Protocols-Algorithms-Source/dp/0471117099/ref=sr\\_1\\_1?ie=UTF8&qid=1287063315&sr=8-1](http://www.amazon.com/Applied-Cryptography-Protocols-Algorithms-Source/dp/0471117099/ref=sr_1_1?ie=UTF8&qid=1287063315&sr=8-1)
- [11] RISC-V International, „riscv/riscv-crypto: RISC-V cryptography extensions standardisation work.“. Zugriffen: 24. Januar 2026. [Online]. Verfügbar unter: <https://github.com/riscv/riscv-crypto>
- [12] kokke und contributors, „kokke/tiny-AES-c: Small portable AES128/192/256 in C“. Zugriffen: 24. Januar 2026. [Online]. Verfügbar unter: <https://github.com/kokke/tiny-AES-c>
- [13] M. Alioto, M. Poli, und S. Rocchi, „Differential Power Analysis Attacks to Precharged Buses: A General Analysis for Symmetric-Key Cryptographic Algorithms“, *IEEE Transactions on Dependable and Secure Computing*, Bd. 7, Nr. 3, S. 226–239, 2010, doi: [10.1109/TDSC.2009.1](https://doi.org/10.1109/TDSC.2009.1) .
- [14] InCore Semiconductors, „RISC-V Compliance Framework (RISCoF) Documentation“. Zugriffen: 20. November 2025. [Online]. Verfügbar unter: <https://riscof.readthedocs.io/en/stable/index.html>
- [15] RISC-V International, „RISC-V Architectural Certification Tests“. Zugriffen: 18. Januar 2026. [Online]. Verfügbar unter: <https://github.com/riscv-non-isa/riscv-arch-test/blob/6ef362b559a749e2f168190c2863ca20f0064517/README.md>

- [16] GitLab Inc., „GitLab Runners | GitLab Docs“. Zugegriffen: 24. Januar 2026. [Online]. Verfügbar unter: <https://docs.gitlab.com/runner/>
- [17] GitLab Inc., „Deployments | GitLab Docs“. Zugegriffen: 24. Januar 2026. [Online]. Verfügbar unter: <https://docs.gitlab.com/ci/environments/deployments/>
- [18] B. I. Ngaleu, „Technische Projektarbeit 1 | Video - Hardware“, technical report, 2025. Zugegriffen: 18. Januar 2026. [Online]. Verfügbar unter: [https://github.com/hsa-ees/piconut/blob/8d4178fc3d8040804ac9da177c024db3701a451a/doc/technical\\_reports/2025-beaurel\\_ingride\\_ngaleu-video\\_in\\_hardware.pdf](https://github.com/hsa-ees/piconut/blob/8d4178fc3d8040804ac9da177c024db3701a451a/doc/technical_reports/2025-beaurel_ingride_ngaleu-video_in_hardware.pdf)
- [19] M. Erichsen, „Entwicklung eines Video-Cores für den PicoNut-Prozessor“, technical report, 2025.
- [20] Mouser Electronics, „LFE5U-85F-6BG381C Lattice | Mouser Europe“. Zugegriffen: 24. Januar 2026. [Online]. Verfügbar unter: <https://eu.mouser.com/ProductDetail/Lattice/LFE5U-85F-6BG381C?qs=Sl%252B5btScQZt9vTUOk2%2FA%3D%3D>

# Erklärung zur Projektarbeit

Hiermit versichere ich, die eingereichte Projektarbeit selbständig verfasst und keine andere als die von mir angegebenen Quellen und Hilfsmittel benutzt zu haben. Wörtlich oder inhaltlich verwendete Quellen wurden entsprechend den anerkannten Regeln wissenschaftlichen Arbeitens zitiert. Ich erkläre weiterhin, dass die vorliegende Arbeit noch nicht anderweitig als Projektarbeit eingereicht wurde. Das Merkblatt zum Täuschungsverbot im Prüfungsverfahren der Technischen Hochschule Augsburg habe ich gelesen und zur Kenntnis genommen. Ich versichere, dass die von mir abgegebene Arbeit keinerlei Plagiate, Texte oder Bilder umfasst, die durch von mir beauftragte Dritte erstellt wurden.

Ausnahme davon sind Abschnitt 3.4 und Abschnitt 3.5 , die von Daniel Sommerfeldt verfasst wurden.

■■■■■■■, den 31.01.2026

Niklas Sirch

Niklas Sirch